FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

Sistema de Control de Inventario y Vencimiento de Medicamentos

Autores:
Galindo Cordero Anabel Mirian (100% participación)
Emilio Valentino Sarmiento Dextre (100% participación)
Renato Gabriel Diaz Carranza (100% participación)

Curso:
Técnicas de programación orientada a objetos

Docente del Curso:
Mg. Jhon Paul Anampa García

Lima – Perú

Modelado del sistema mediante UML

Parte 1: Diagrama de clases del proyecto FARMASTOCK

1. Contexto del proyecto

FARMASTOCK es un sistema pensado para las farmacias de barrio pequeñas que hoy llevan su inventario de medicamentos en cuadernos de registro u hojas de cálculo aisladas. Esa forma de trabajo dificulta saber con precisión qué productos están próximos a vencer, cuántas unidades hay disponibles de cada referencia y cuáles ya no deberían venderse por haber superado su fecha de vencimiento.

Con base en esos requerimientos, este modelo de clases se elaboró a partir del código del repositorio del proyecto (carpeta src) y cubre tres necesidades del sistema:

- Controlar el stock disponible de cada medicamento.
- Detectar los productos próximos a vencer.
- Impedir la venta de productos vencidos o sin stock suficiente.

2. Identificación de clases principales

El proyecto tiene seis clases: una clase de arranque (Main), un controlador (InventarioController), una clase de dominio (Medicamento) y tres excepciones propias que protegen las reglas de negocio. Las tres excepciones heredan de la clase Exception de Java.

Clase	Descripción	Responsabilidad principal
Main	Clase de arranque del programa. Demuestra el flujo completo del sistema por consola.	Registrar medicamentos de prueba, ejecutar ventas, consultas y reportes, y capturar las excepciones mostrando el motivo del rechazo.
InventarioController	Controlador del inventario. Guarda la colección de medicamentos y concentra la lógica del negocio.	Agregar y buscar medicamentos, procesar ventas con validaciones, listar los próximos a vencer y agruparlos por categoría.
Medicamento	Producto de la farmacia con su stock y su fecha de vencimiento.	Guardar los datos del producto y saber si está vencido o si vence dentro de un número de días.
MedicamentoNoEncontradoException	Excepción propia (hereda de Exception).	Avisar que una búsqueda por id o por nombre no encontró coincidencias.
MedicamentoVencidoException	Excepción propia (hereda de Exception).	Impedir la venta de un medicamento cuya fecha de vencimiento ya pasó.
StockInsuficienteException	Excepción propia (hereda de Exception).	Avisar que la cantidad pedida supera el stock disponible.

3. Atributos relevantes

Clase	Atributos
Main	(no declara atributos)
InventarioController	- inventario : ArrayList<Medicamento> - FORMATO_FECHA : DateTimeFormatter (formato yyyy-MM-dd)
Medicamento	- id : int - nombre : String - categoria : String - precio : double - cantidad : int - fechaVencimiento : LocalDate
MedicamentoNoEncontradoException	(sin atributos propios; hereda de Exception)
MedicamentoVencidoException	(sin atributos propios; hereda de Exception)
StockInsuficienteException	(sin atributos propios; hereda de Exception)

4. Métodos generales

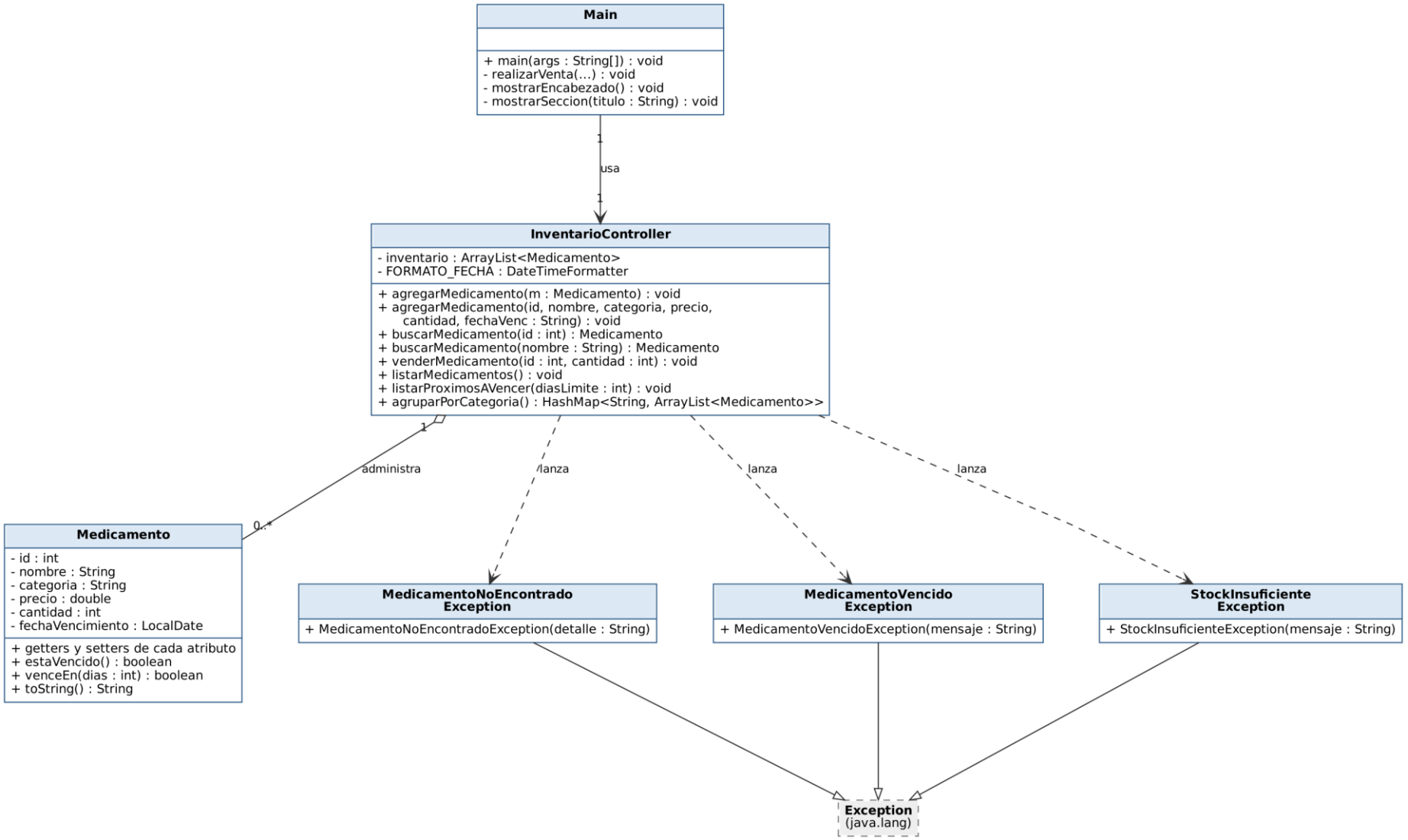
Clase	Métodos
Main	+ main(args : String[]) : void + realizarVenta(...) : void + mostrarEncabezado() : void + mostrarSeccion(titulo : String) : void
InventarioController	+ agregarMedicamento(m : Medicamento) : void + agregarMedicamento(id, nombre, categoria, precio, cantidad, fechaVenc : String) : void + buscarMedicamento(id : int) : Medicamento + buscarMedicamento(nombre : String) : Medicamento + venderMedicamento(id : int, cantidad : int) : void + listarMedicamentos() : void + listarProximosAVencer(diasLimite : int) : void + agruparPorCategoria() : HashMap<String, ArrayList<Medicamento>>
Medicamento	+ constructor con los seis atributos + getters y setters de cada atributo + estaVencido() : boolean + venceEn(dias : int) : boolean + toString() : String
MedicamentoNoEncontradoException	+ MedicamentoNoEncontradoException(detalle : String)
MedicamentoVencidoException	+ MedicamentoVencidoException(mensaje : String)
StockInsuficienteException	+ StockInsuficienteException(mensaje : String)

Nota: los métodos de Main y los de InventarioController se resumen a nivel general; sus parámetros exactos pueden variar ligeramente respecto al código.

5. Relaciones preliminares entre clases

Origen	Tipo	Destino	Multiplicidad	Descripción
Main	Asoc. dirigida	InventarioController	1 a 1	Main crea y usa el controlador, y captura sus excepciones.
InventarioController	Agregación	Medicamento	1 a 0..*	El controlador administra una lista de medicamentos.
InventarioController	Dependencia	MedicamentoNoEncontradoException	—	Lanza la excepción cuando la búsqueda no encuentra el producto.
InventarioController	Dependencia	MedicamentoVencidoException	—	La lanza al intentar vender un producto vencido.
InventarioController	Dependencia	StockInsuficienteException	—	La lanza si la cantidad pedida supera el stock.
MedicamentoNoEncontradoException	Herencia	Exception	—	Es una excepción verificada de Java.
MedicamentoVencidoException	Herencia	Exception	—	Es una excepción verificada de Java.
StockInsuficienteException	Herencia	Exception	—	Es una excepción verificada de Java.

6. Diagrama de clases UML



https://github.com/AnabelGalindo/FARMACIA/